CS342 · VISUAL PROGRAMMING

Final Drill Practice

All the rules, then 8 questions in new forms within the midterm scope

Muharib

01 OOP rules

Topic	The rule
Constructor order	new C() runs the grandparent's constructor, then the parent's, then the child's. Every constructor calls super() automatically on its first line.
Which method runs	By the **real object** (after new), even when it is called from inside a parent method.
Which methods you may call	By the **variable type** (before =). Animal a = new Dog(); then a.fetch(); is a compile error because Animal has no fetch.
super.show()	Runs the parent's version, then continues the rest of the child's method.
Downcast	Always compiles. If the real object is not of that type: ClassCastException at run time and the program stops.
abstract · interface · final	An abstract class cannot be created with new. A class that implements an interface must implement all its methods. A final class cannot be inherited.

02 Exception rules

Topic	The rule
try	The first exception skips the rest of the try and jumps to the matching catch.
catch	Read top to bottom; the first match runs. Putting Exception before one of its children (such as NumberFormatException) is a compile error.
finally	Runs **always**, with or without an exception.
An uncaught exception	The program stops at that line; nothing after it prints.
Integer.parseInt	"abc", " " and "2.3" all throw NumberFormatException.
A custom exception	class X extends Exception. The text given to super(msg) is returned by getMessage(); toString() is what println(e) prints. A method that throws it must declare throws X.

03 Window rules

This template is fixed. Only the components, the layout, the size and the title change.

```
import javax.swing.*;
import java.awt.*;
public class MyWindow extends JFrame {
    private JLabel l;                                   // components as attributes
    private JButton b;
    public MyWindow() {
        super("Title");                                 // the window title
        setSize(300, 150);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        JPanel p = (JPanel) getContentPane();
        p.setLayout(new FlowLayout());
        l = new JLabel("Hello");
        b = new JButton("OK");
        p.add(l);
        p.add(b);
    }
    public static void main(String[] args) {
        MyWindow f = new MyWindow();
        f.setVisible(true);
    }
}
```

Layout	How it arranges	Default for
FlowLayout	One after another on a line, each at its natural size; wraps when the line is full.	JPanel
BorderLayout	5 regions: NORTH, SOUTH, EAST, WEST and CENTER. add without a region goes to CENTER.	The JFrame content pane
GridLayout(r, c)	A table of r rows and c columns; equal cells filled left to right, then the next row.	—

Reading a window picture: rows stacked vertically? GridLayout(n, 1), each row a JPanel. Something at the top, the bottom and the middle? BorderLayout. Everything on one line? FlowLayout.

04 Questions

Q1 FIND OUTPUT · CONSTRUCTORS

Find the output of the program below.

```
class A {
    A() { System.out.println("A()"); }
    void show() { System.out.println("A.show"); }
}
class B extends A {
    B() { System.out.println("B()"); }
    void show() { super.show(); System.out.println("B.show"); }
}
class C extends B {
    C() { super(); System.out.println("C()"); }
}
public class Q1 {
    public static void main(String[] args) {
        A x = new C();
        x.show();
        A z = new A();
        z.show();
    }
}
```

Q1 OUTPUT

Output:

Q2 FIND OUTPUT · CASTING

Find the output of the program below.

```
class Animal {
    String name() { return "Animal"; }
    void speak() { System.out.println(name() + " makes a sound"); }
}
class Dog extends Animal {
    String name() { return "Dog"; }
    void fetch() { System.out.println("Dog fetches"); }
}
class Cat extends Animal {
    void speak() { System.out.println("Cat meows"); }
}
public class Q2 {
    public static void main(String[] args) {
        Animal[] a = { new Dog(), new Cat(), new Animal() };
        for (int i = 0; i < a.length; i++)
            a[i].speak();
        ((Dog) a[0]).fetch();
        ((Dog) a[1]).fetch();
        System.out.println("End");
    }
}
```

Q2 OUTPUT

Output:

Q3 COMPILE OR NOT

Using the classes below, say for each line (a) to (h): **OK**, **compile error** or **run-time error**.

```
abstract class Shape { abstract double area(); }
interface Flyable { void fly(); }
class Animal { void speak() { } }
class Dog extends Animal { void fetch() { } }
final class X { }
(a)  Shape s = new Shape();
(b)  class Bird implements Flyable { }
(c)  Object o = "hi";  Integer n = (Integer) o;
(d)  Animal x = new Dog();  x.fetch();
(e)  Animal x = new Dog();  Dog d = (Dog) x;  d.fetch();
(f)  class Y extends X { }
(g)  int n = Integer.parseInt("12a");
(h)  try { ... } catch (Exception e) { } catch (NumberFormatException e) { }
```

Q3 ANSWERS

(a) _____ (b) _____ (c) _____ (d) _____
(e) _____ (f) _____ (g) _____ (h) _____

Q4 FIND OUTPUT · EXCEPTIONS

Find the output of the program below.

```
class TooBigException extends Exception {
    TooBigException(String s) { super(s); }
}
public class Q4 {
    static int check(String s) throws TooBigException {
        int n = Integer.parseInt(s);
        if (n > 50) throw new TooBigException("too big: " + n);
        return n * 2;
    }
    public static void main(String[] args) {
        String[] in = {"10", "abc", "70", "5"};
        int total = 0;
        for (int i = 0; i < in.length; i++) {
            try {
                total += check(in[i]);
                System.out.println("ok " + in[i]);
            }
            catch (NumberFormatException e) {
                System.out.println("not a number");
            }
            catch (TooBigException e) {
                System.out.println(e.getMessage());
            }
            finally {
                System.out.println("done " + i);
            }
        }
        System.out.println("total = " + total);
    }
}
```

Q4 OUTPUT

Output:

Q5 WRITE CODE · CUSTOM EXCEPTION

(a) Write the exception class AgeException whose toString() returns "Invalid age".
(b) Write the class Person with an attribute age and a method setAge(int a) that throws AgeException when a < 0 or a > 120.
(c) In main, call setAge(30) then setAge(150), and print the exception if it happens. What is the output?

Q6 DRAW THE WINDOW

(a) Draw the window displayed by the program below.

(b) What is the layout of the content pane p, and why is setLayout not called on it?

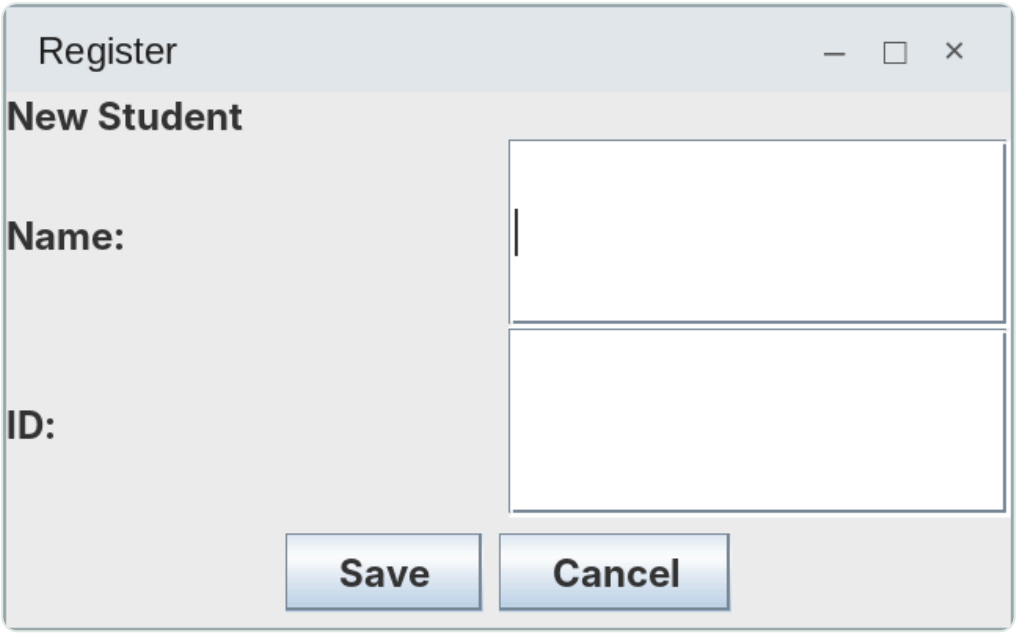
```
import javax.swing.*;
import java.awt.*;
public class Q6 extends JFrame {
    public Q6() {
        super("Q6");
        JPanel p = (JPanel) getContentPane();
        p.add(new JButton("Up"), BorderLayout.NORTH);
        p.add(new JButton("Down"), BorderLayout.SOUTH);
        p.add(new JButton("Left"), BorderLayout.WEST);
        p.add(new JButton("Right"), BorderLayout.EAST);
        JPanel c = new JPanel(new GridLayout(2, 2));
        c.add(new JButton("1"));
        c.add(new JButton("2"));
        c.add(new JButton("3"));
        c.add(new JButton("4"));
        p.add(c, BorderLayout.CENTER);
        setSize(300, 200);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
    public static void main(String[] args) {
        new Q6().setVisible(true);
    }
}
```

Q6 DRAWING

(a) The window: (b) below the grid.

Q7 WRITE THE WINDOW

Write the Java class Q7 that displays the window below (title "Register"). Use the BorderLayout of the content pane: the label "New Student" at the north, a JPanel with a GridLayout(2, 2) in the center, and a JPanel holding the two buttons at the south. All components must be attributes of the class.



Q7 CODE

Code:

Q8 MCQ

1. The default layout of the JFrame content pane is:

a) FlowLayout b) BorderLayout c) GridLayout d) No layout

2. The default layout of a JPanel is:

a) FlowLayout b) BorderLayout c) GridLayout d) No layout

3. new GridLayout(2, 3) creates:

a) 2 rows and 3 columns b) 3 rows and 2 columns c) 2 cells d) 6 rows

4. The Swing components are in the package:

a) java.awt b) java.swing c) javax.swing d) javax.awt

5. The method pack():

a) Shows the window b) Closes the window c) Sets the layout d) Sizes the window to fit its components

6. Swing components (except JFrame, JDialog, JWindow and JApplet) are:

a) Heavyweight b) Lightweight c) Part of AWT d) OS-dependent